

COMPTE RENDU

TD2 – La persistance des données en PHP (BDD, PDO)

Délio Duchesne-Brajon

BTS SIO 2ème année – option SLAM4

Projet local : C:\xampp\htdocs\mvc

1. Connexion à la base de données

Exercice 1 – Table voiture

J'ai créé la table voiture dans phpMyAdmin (base mvc) avec les 3 champs demandés : immatriculation en VARCHAR(8) comme clé primaire, marque en VARCHAR(25) et couleur en VARCHAR(12), en InnoDB et utf8_general_ci. J'ai ensuite inséré deux voitures avec l'onglet Insérer, et vérifié le contenu avec un SELECT * FROM voiture dans la console SQL (voir les captures ci-dessous, avec au passage le panneau de contrôle XAMPP où l'on voit que mon MySQL tourne sur le port 3307).

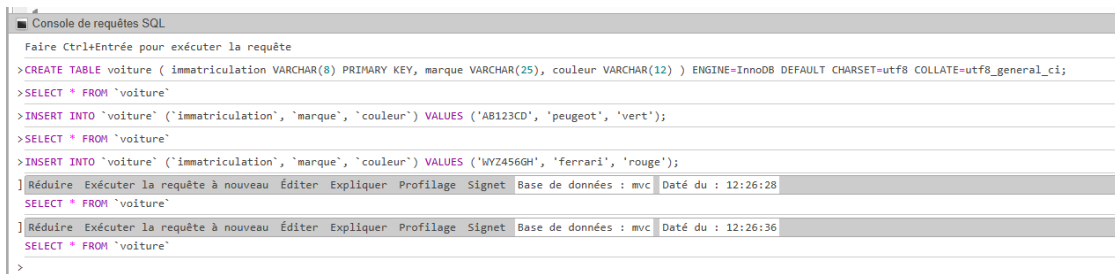
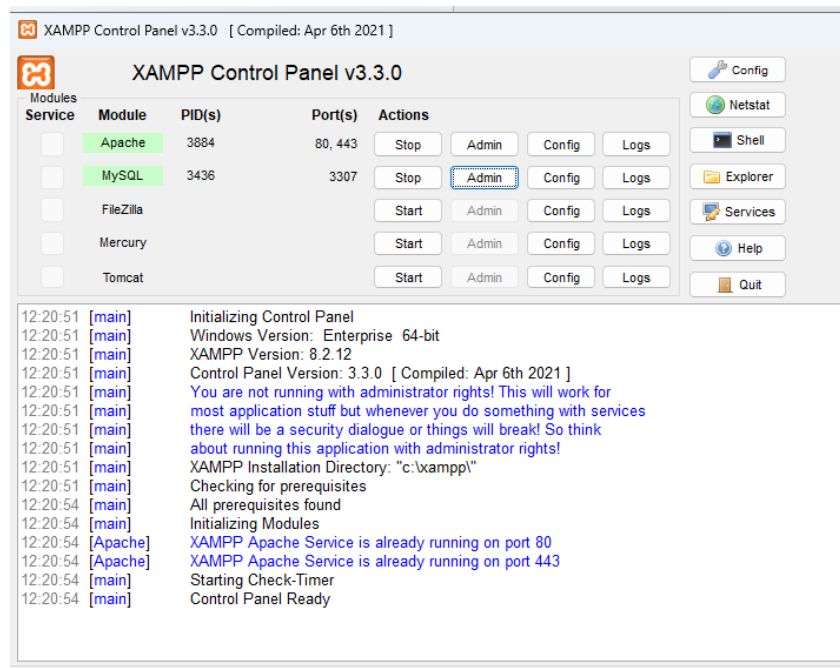
The screenshot shows the phpMyAdmin interface for the 'mvc' database. The 'voiture' table is selected, and its structure is displayed. The table has three columns: 'immatriculation' (VARCHAR(8), PRIMARY KEY), 'marque' (VARCHAR(25)), and 'couleur' (VARCHAR(12)). The table is using the InnoDB engine and the utf8_general_ci collation. Below the structure, the 'Insérer' (Insert) tab is active, showing two rows of data:

immatriculation	marque	couleur
AB123CD	peugeot	vert
WY2456GH	ferrari	rouge

Below the table, the 'Console de requêtes SQL' (SQL Query Console) is visible, showing the SQL commands used to create the table and insert data:

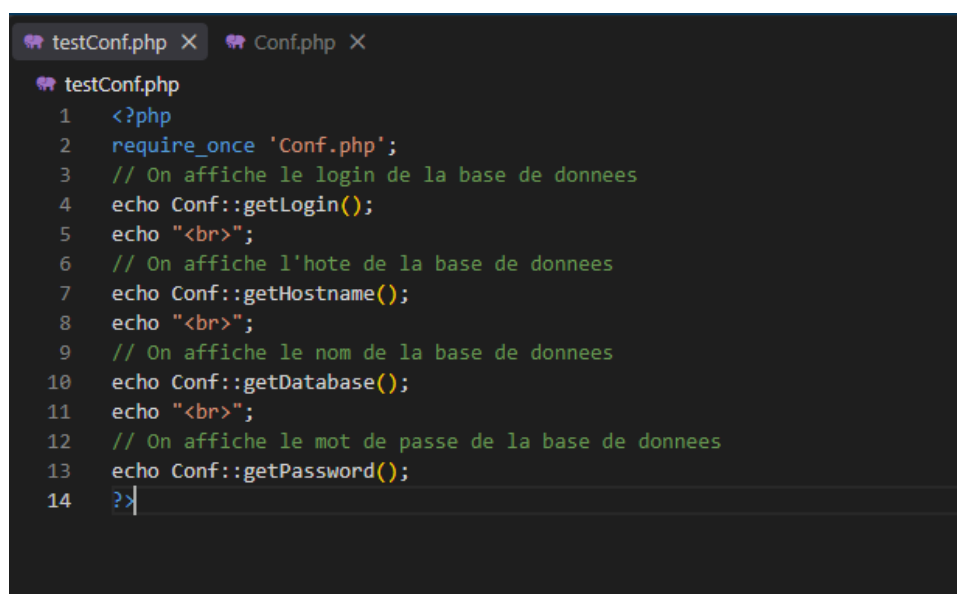
```

>CREATE TABLE voiture ( immatriculation VARCHAR(8) PRIMARY KEY, marque VARCHAR(25), couleur VARCHAR(12) ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_general_ci;
>SELECT * FROM 'voiture'
>INSERT INTO 'voiture' ('immatriculation', 'marque', 'couleur') VALUES ('AB123CD', 'peugeot', 'vert');
>INSERT INTO 'voiture' ('immatriculation', 'marque', 'couleur') VALUES ('WY2456GH', 'ferrari', 'rouge');
>SELECT * FROM 'voiture'
>SELECT * FROM 'voiture'
  
```



Exercice 2 – Conf.php

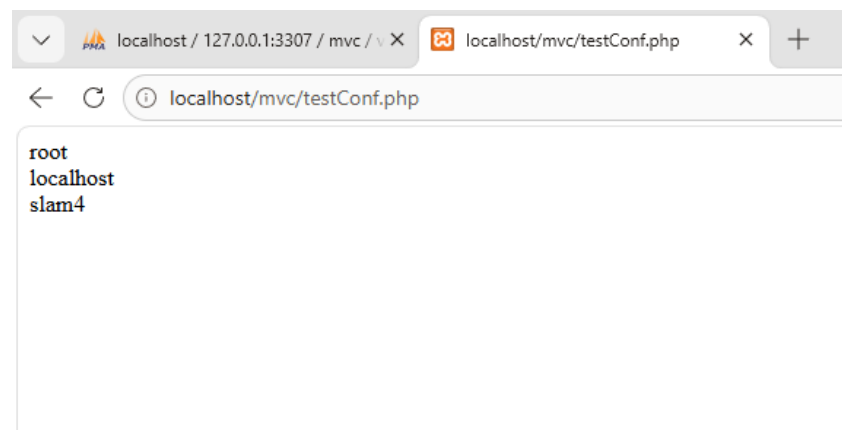
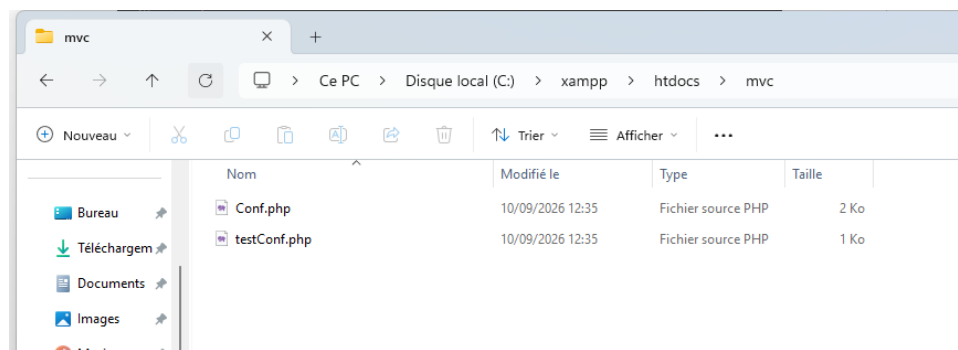
J'ai créé le fichier Conf.php avec la classe Conf, son attribut statique \$databases (hostname, database, login, password) et les méthodes statiques getLogin(), getHostname(), getDatabase() et getPassword(). Sur ma machine, la base s'appelle slam4 et le compte est root sans mot de passe. J'ai testé la classe avec testConf.php : la page affiche bien root, localhost puis slam4 dans le navigateur, ce qui confirme que les getters fonctionnent.



```

testConf.php X Conf.php X
Conf.php
1 <?php
2 class Conf {
3
4     static private $databases = array(
5         // ou localhost sur votre machine
6         'hostname' => 'localhost',
7         // A l'IUT, vous avez une BDD nommée comme votre login
8         // Sur votre machine, vous devrez créer une BDD
9         'database' => 'slam4',
10        // A l'IUT, c'est votre login
11        // Sur votre machine, vous avez sûrement un compte 'root'
12        'login' => 'root',
13        // A l'IUT, c'est votre mdp (INE par défaut)
14        // Sur votre machine personnelle, vous avez créé ce mdp à l'installation
15        'password' => ''
16    );
17
18    // la variable debug est un boolean
19    static private $debug = true;
20
21    static public function getLogin() {
22        // en PHP l'indice d'un tableau n'est pas forcément un chiffre.
23        return self::$databases['login'];
24    }
25
26    static public function getHostname() {
27        return self::$databases['hostname'];
28    }
29
30    static public function getDatabase() {
31        return self::$databases['database'];
32    }
33
34    static public function getPassword() {
35        return self::$databases['password'];
36    }
37
38    static public function getDebug() {
39        return self::$debug;
40    }
41
42 }
43 ?>

```



Exercice 3 – Git

Déjà fait auparavant (voir capture du commit à l'exercice 6).

Exercice 4 – Model.php et PDO

J'ai créé Model.php avec la classe Model, son attribut statique \$pdo et la fonction Init() qui récupère les informations de connexion via Conf puis crée l'objet PDO (en précisant le port 3307 utilisé par mon MySQL local, sinon la connexion échoue). J'appelle Model::Init() à la fin du fichier pour que la connexion soit initialisée dès l'inclusion de la classe. Le test avec testModel.php affiche bien « Connexion réussie ! ».

```

1 <?php
2 require_once 'Conf.php';
3
4 class Model {
5     public static $pdo;
6
7     public static function Init() {
8         $hostname = Conf::getHostname();
9         $database_name = Conf::getDatabase();
10        $login = Conf::getLogin();
11        $password = Conf::getPassword();
12
13        self::$pdo = new PDO("mysql:host=$hostname;port=3307;dbname=$database_name", $login, $password);
14    }
15 }
16
17 Model::Init();
18 ?>

```



Exercice 5 – Gestion des erreurs

J'ai placé le new PDO(...) dans un bloc try/catch(PDOException \$e) pour récupérer proprement les erreurs de connexion (affichage du message puis die()). J'ai aussi mis à jour la connexion pour forcer l'UTF-8 avec PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8", et activé le mode d'erreurs par exception avec self::\$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION).

```

15 }
16 try {
17     self::$pdo = new PDO("mysql:host=$hostname;port=3307;dbname=$database_name", $login, $password);
18 } catch (PDOException $e) {
19     echo $e->getMessage(); // affiche un message d'erreur
20     die();
21 }
22 Model::Init();
23 ?>

```

```

12         }
13         self::$pdo = new PDO("mysql:host=$hostname;port=3307;dbname=$database_name", $login, $password);
14     }
15 }
16 try {
17     self::$pdo = new PDO("mysql:host=$hostname;port=3307;dbname=$database_name", $login, $password);
18 } catch (PDOException $e) {
19     echo $e->getMessage(); // affiche un message d'erreur
20     die();
21 } self::$pdo = new PDO(
22     "mysql:host=$hostname;port=3307;dbname=$database_name",
23     $login,
24     $password,
25     array(PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8")
26 );
27 self::$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
28 Model::Init();
29 ?>

```

```

1  <?php
2  require_once 'Conf.php';
3
4  class Model {
5      public static $pdo;
6
7      public static function Init() {
8          $hostname = Conf::getHostname();
9          $database_name = Conf::getDatabase();
10         $login = Conf::getLogin();
11         $password = Conf::getPassword();
12
13         try {
14             self::$pdo = new PDO(
15                 "mysql:host=$hostname;port=3307;dbname=$database_name",
16                 $login,
17                 $password,
18                 array(PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8")
19             );
20             self::$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
21         } catch (PDOException $e) {
22             echo $e->getMessage(); // affiche un message d'erreur
23             die();
24         }
25     }
26 }
27
28 Model::Init();
29 ?>

```

2. Opérations sur la base de données

Exercice 6 – FETCH_OBJ

J'ai créé lireVoiture.php : il inclut Model.php, exécute `SELECT * FROM voiture` avec `Model::$pdo->query()`, puis récupère le résultat avec `fetchAll(PDO::FETCH_OBJ)`. J'affiche ensuite chaque voiture dans une boucle `foreach` en accédant aux attributs `marque`, `couleur` et `immatriculation` des objets renvoyés. J'en ai profité pour initialiser Git sur le projet (`git init`, configuration du nom et de l'email) et faire mon premier commit.

```

trust this folder to enable all features. Manage Learn More
testModel.php X Model.php X lireVoiture.php X Conf.php X
lireVoiture.php
1 <?php
2 require_once "Model.php";
3
4 $rep = Model::$pdo->query("SELECT * FROM voiture");
5 $tab_obj = $rep->fetchAll(PDO::FETCH_OBJ);
6
7 foreach ($tab_obj as $obj) {
8     echo $obj->marque . " " . $obj->couleur . " (" . $obj->immatriculation . ")<br>";
9 }
10 ?>

```

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\xampp\htdocs\mvc> git status
fatal: not a git repository (or any of the parent directories): .git
PS C:\xampp\htdocs\mvc> git init
Initialized empty Git repository in C:/xampp/htdocs/mvc/.git/
PS C:\xampp\htdocs\mvc> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    Conf.php
    Model.php
    lireVoiture.php
    testConf.php
    testModel.php

nothing added to commit but untracked files present (use "git add" to track)
PS C:\xampp\htdocs\mvc> git config --global user.name "Delio Duchesne-Brajon"
PS C:\xampp\htdocs\mvc> git config --global user.email "deliodcb@gmail.com"
PS C:\xampp\htdocs\mvc> git add .
PS C:\xampp\htdocs\mvc> git commit -m "Premier commit - TD2 PDO"
[master (root-commit) d100682] Premier commit - TD2 PDO
 5 files changed, 101 insertions(+)
 create mode 100644 Conf.php
 create mode 100644 Model.php
 create mode 100644 lireVoiture.php
 create mode 100644 testConf.php
 create mode 100644 testModel.php
PS C:\xampp\htdocs\mvc> git log
commit d1006826291539fd796f72bd82db850afa341db (HEAD -> master)
Author: Delio Duchesne-Brajon <deliodcb@gmail.com>
Date: Thu Sep 10 13:25:47 2026 +0200

```

Exercice 7 – FETCH_CLASS

J'ai remplacé FETCH_OBJ par FETCH_CLASS pour que PDO crée directement des objets Voiture : \$rep->setFetchMode(PDO::FETCH_CLASS, 'Voiture') puis \$rep->fetchAll(). Pour que ça fonctionne, j'ai adapté le constructeur de Voiture pour qu'il accepte aussi un appel sans arguments. Dans lireVoiture.php, l'affichage se fait maintenant avec \$voit->afficher() dans le foreach, et le résultat dans le navigateur est identique à avant (peugeot vert et ferrari rouge). J'ai commité ce changement en expliquant la différence avec FETCH_OBJ dans le message de commit.

```

lireVoiture.php
1  <?php
2  require_once "Model.php";
3  require_once "Voiture.php";
4
5  $rep = Model::$pdo->query("SELECT * FROM voiture");
6  $rep->setFetchMode(PDO::FETCH_CLASS, 'Voiture');
7  $tab_voit = $rep->fetchAll();
8
9  foreach ($tab_voit as $voit) {
10     $voit->afficher();
11 }
12 ?>

```

```

Premier commit - 102 PDO
PS C:\xampp\htdocs\mvc> git add
PS C:\xampp\htdocs\mvc> git commit -m "Ici on utilise PDO::FETCH_CLASS à la place de FETCH_OBJ : PDO va créer directement des instances de Voiture, remplir leurs attributs avec les champs de la BDD, puis appeler le constructeur sans arguments (d'où le constructeur flexible __construct($m = NULL, $c = NULL, $i = NULL) qu'on a déjà mis dans Voiture.php)."
[master 3aca942] Ici on utilise PDO::FETCH_CLASS à la place de FETCH_OBJ : PDO va créer directement des instances de Voiture, remplir leurs attributs avec les champs de la BDD, puis appeler le constructeur sans arguments (d'où le constructeur flexible __construct( = NULL, = NULL, = NULL) qu'on a déjà mis dans Voiture.php).
1 file changed, 5 insertions(+), 3 deletions(-)
PS C:\xampp\htdocs\mvc>

```

Exercice 8 – getAllVoitures()

J'ai isolé le code de récupération des voitures dans une méthode statique `getAllVoitures()` de la classe `Voiture`, qui fait la requête et le `fetchAll(PDO::FETCH_CLASS)` en interne. `lireVoiture.php` appelle maintenant directement `Voiture::getAllVoitures()`, ce qui rend le fichier beaucoup plus court et plus lisible.

```

6
7     public static function getAllVoitures() {
8         $rep = Model::$pdo->query("SELECT * FROM voiture");
9         $rep->setFetchMode(PDO::FETCH_CLASS, 'Voiture');
10        return $rep->fetchAll();
11    }

```

```

lireVoiture.php
1  <?php
2  require_once "Model.php";
3  require_once "Voiture.php";
4
5  $tab_voit = Voiture::getAllVoitures();
6
7  foreach ($tab_voit as $voit) {
8      $voit->afficher();
9  }
10 ?>

```

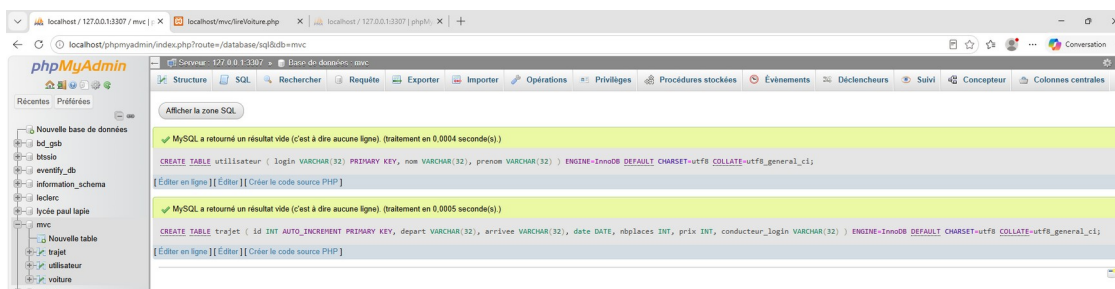
Exercice 9 – Mode debug

Ajouté directement dans Conf.php (voir exercice 2).

3. Site de covoiturage

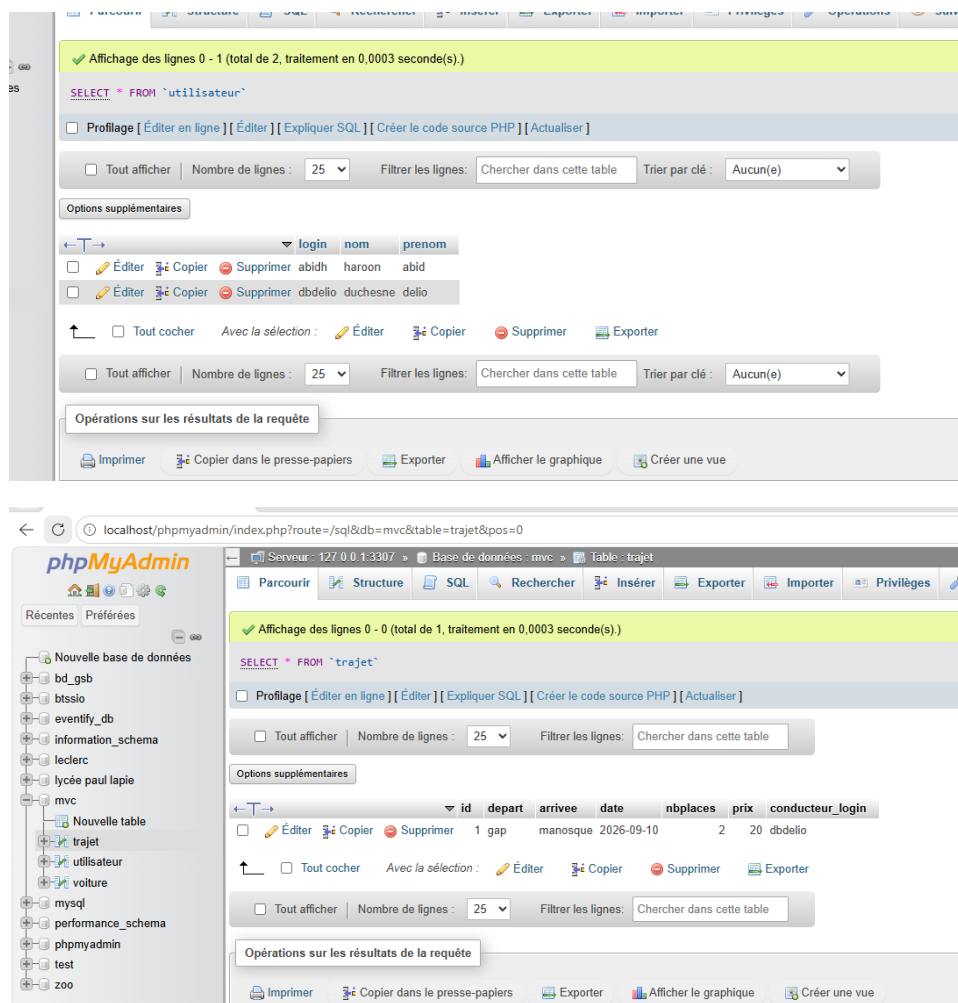
1/ Création des tables

Dans phpMyAdmin, j'ai créé les tables utilisateur (login en VARCHAR(32) comme clé primaire, nom et prénom en VARCHAR(32)) et trajet (id en INT auto-incrémenté comme clé primaire, depart, arrivee, date, nbplaces, prix et conducteur_login), toujours en InnoDB et utf8_general_ci comme demandé.



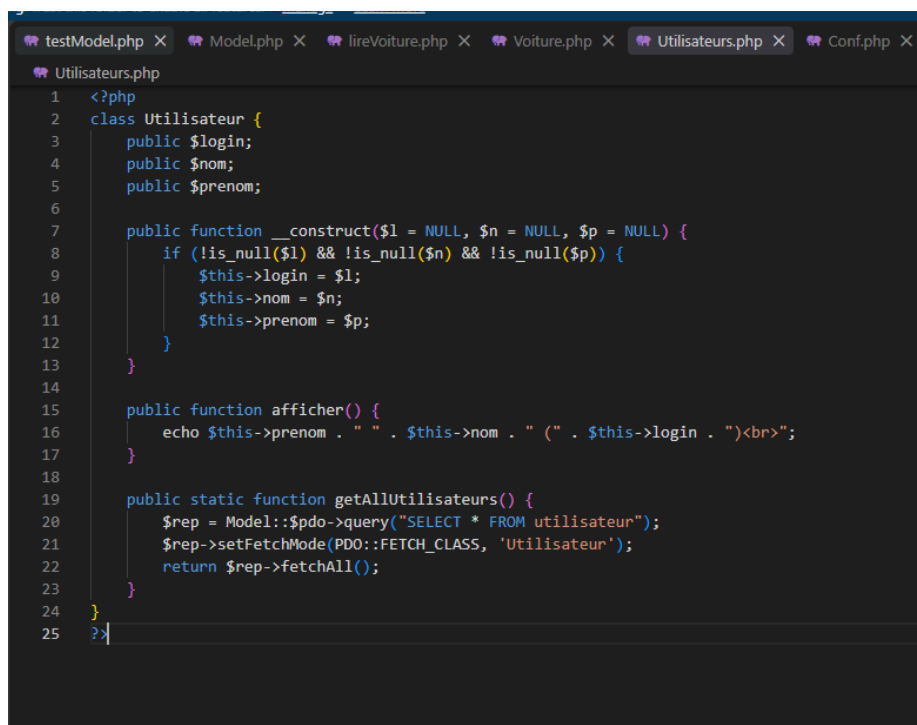
2/ Données utilisateur et trajet

J'ai inséré deux utilisateurs dans la table utilisateur et un trajet dans la table trajet, en faisant attention à ne pas remplir le champ id (pour laisser l'auto-incrément faire son travail) et à mettre dans conducteur_login le login d'un utilisateur qui existe bien (dbdelio).



3/ Classes Utilisateur et Trajet

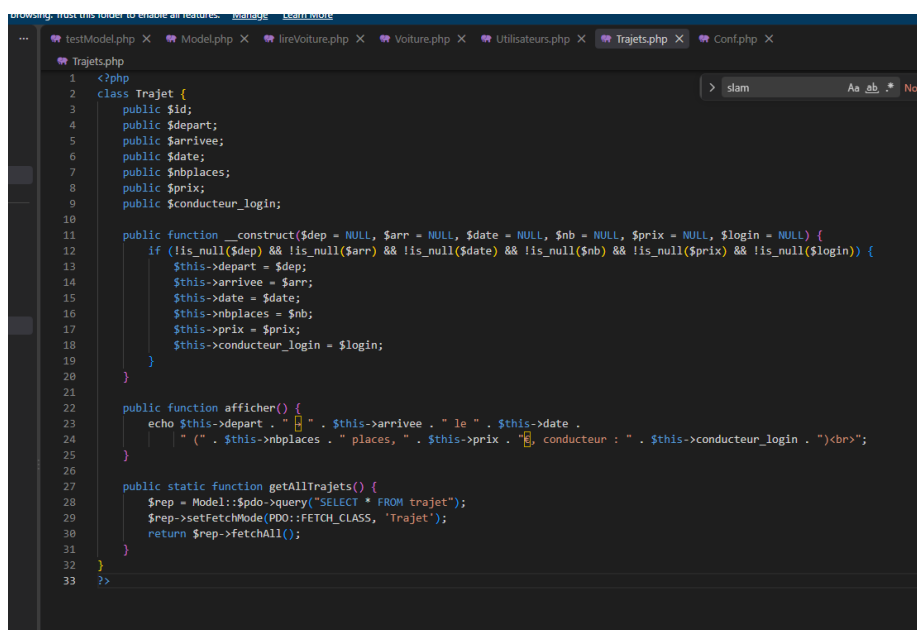
J'ai repris les classes Utilisateur et Trajet du TD précédent sur le covoiturage et je les ai adaptées comme pour Voiture : constructeur avec des paramètres optionnels (NULL par défaut) pour accepter un appel sans arguments, méthode afficher(), et méthode statique getAllUtilisateurs() / getAllTrajets() qui utilisent PDO::FETCH_CLASS pour renvoyer directement des tableaux d'objets.



```

1  <?php
2  class Utilisateur {
3      public $login;
4      public $nom;
5      public $prenom;
6
7      public function __construct($l = NULL, $n = NULL, $p = NULL) {
8          if (!is_null($l) && !is_null($n) && !is_null($p)) {
9              $this->login = $l;
10             $this->nom = $n;
11             $this->prenom = $p;
12         }
13     }
14
15     public function afficher() {
16         echo $this->prenom . " " . $this->nom . " (" . $this->login . ")<br>";
17     }
18
19     public static function getAllUtilisateurs() {
20         $rep = Model::$pdo->query("SELECT * FROM utilisateur");
21         $rep->setFetchMode(PDO::FETCH_CLASS, 'Utilisateur');
22         return $rep->fetchAll();
23     }
24 }
25 ?>

```



```

1  <?php
2  class Trajet {
3      public $id;
4      public $depart;
5      public $arrivee;
6      public $date;
7      public $nbplaces;
8      public $prix;
9      public $conducteur_login;
10
11     public function __construct($dep = NULL, $arr = NULL, $date = NULL, $nb = NULL, $prix = NULL, $login = NULL) {
12         if (!is_null($dep) && !is_null($arr) && !is_null($date) && !is_null($nb) && !is_null($prix) && !is_null($login)) {
13             $this->depart = $dep;
14             $this->arrivee = $arr;
15             $this->date = $date;
16             $this->nbplaces = $nb;
17             $this->prix = $prix;
18             $this->conducteur_login = $login;
19         }
20     }
21
22     public function afficher() {
23         echo $this->depart . " " . $this->arrivee . " le " . $this->date .
24             " " . $this->nbplaces . " places, " . $this->prix . "€, conducteur : " . $this->conducteur_login . "<br>";
25     }
26
27     public static function getAllTrajets() {
28         $rep = Model::$pdo->query("SELECT * FROM trajet");
29         $rep->setFetchMode(PDO::FETCH_CLASS, 'Trajet');
30         return $rep->fetchAll();
31     }
32 }
33 ?>

```

5/ 6/ Pages de lecture

J'ai créé lireUtilisateurs.php et lireTrajets.php sur le même modèle que lireVoiture.php : ils incluent Model.php et la classe correspondante, appellent la méthode getAllUtilisateurs() ou getAllTrajets(), puis affichent chaque objet avec une boucle foreach et la méthode afficher().

```

1 <?php
2 require_once "Model.php";
3 require_once "Utilisateur.php";
4
5 foreach (Utilisateur::getAllUtilisateurs() as $u) {
6     $u->afficher();
7 }
8 ?>

```

```

1 <?php
2 require_once "Model.php";
3 require_once "Trajet.php";
4
5 foreach (Trajet::getAllTrajets() as $t) {
6     $t->afficher();
7 }
8 ?>

```

7/ Commit final

Pour finir, j'ai enregistré tout mon travail avec Git (git add . puis git commit) avec un message qui résume cette dernière partie : tables utilisateur/trajet et classes correspondantes.

```

3 arguments (0 ou 1 constructeur flexible __construct = NULL, = NULL, = NULL) qu'on a déjà mis dans voiture.php
1 file changed, 5 insertions(+), 3 deletions(-)
PS C:\xampp\htdocs\mvc> git add .
PS C:\xampp\htdocs\mvc> git commit -m "Partie 3 - covoiturage : tables utilisateur/trajet + classes"
[master 12dc9d4] Partie 3 - covoiturage : tables utilisateur/trajet + classes
6 files changed, 102 insertions(+), 3 deletions(-)
create mode 100644 Trajets.php
create mode 100644 Utilisateurs.php
create mode 100644 Voiture.php
create mode 100644 lireTrajets.php
create mode 100644 lireUtilisateurs.php
PS C:\xampp\htdocs\mvc> |

```